

Image Data Colouring

Business Objective

Image colorization is the process of taking an input grayscale (black and white) image and then producing an output colorized image that represents the semantic colors and tones of the input. In image colorization, a color is assigned to each pixel of a target grayscale image.

Image colorization technique is helpful for many applications such as Medical Microscope, Medical Imagery, Denoising and recreating old Images, Night Vision Camera, etc.

In our use case, we will be using a fully automated data-driven technique called autoencoders for image colorization. Autoencoders are a specific type of feedforward neural network where the input is the same as the output. The VGG16 model will be used as a feature extractor. VGG16 is a classic neural network used as a backbone for many computer vision tasks

This project aims to build a convolutional neural network that will best convert the grayscale images to RGB images.

Data Description

Here, we will be using a landscape image dataset which consists of approximately,

- Training - 7000 images (RGB images)
- Testing - 5 images (Gray scale images)

Aim

To build a model using keras, that can best convert a grayscale image to a coloured image.

Tech stack

- Language - Python
- Libraries - numpy, pandas, tensorflow, keras

Approach

1. Mount the drive for loading the input data images.
2. Import the necessary libraries.
3. Import and initialize the VGG16 model
4. Initialise the ImageDataGenerator to rescale the images.
5. Convert the RGB to LAB format
6. Create a sequential model and check its summary
7. Compile the model with the appropriate optimizer, loss, and performance metric
8. Fit the model and train it on 2000 epochs with a defined batch size
9. Save the model
10. Using the saved model, make predictions on the test images
11. Check the predicted test images

Modular code overview

Input

- |_Training images
- |_Test images

Src

- |_Engine.py
- |_ML_Pipeline
 - |_LoadData.py
 - |_EncoderLayers.py
 - |_DecoderLayers.py
 - |_Inference.py
 - |_References.py

Lib

- |_Image_Coloring_using_Transfer_Learning.ipynb

Output

- |_colorize_autoencoder_2000.model
- |_generated images

Once you unzip the modular_code.zip file you can find the following folders within it.

1. Input
2. Src
3. Output
4. Lib

1. Input folder - It contains all the data that we have for analysis. There are two data files in our case,
 - Training data folder (approx. 7000 images)
 - Test data folder (5 images)
2. Src folder - This is the most important folder of the project. This folder contains all the modularized code for all the above steps in a modularized manner. This folder consists of:
 - Engine.py
 - ML_Pipeline

The ML_Pipeline is a folder that contains all the functions put into different python files which are appropriately named. These python functions are then called inside the engine.py file.
3. Output folder – The output folder contains the best fitted model that we trained for this data. This model can be easily loaded and used for future use and the user need not have to train all the models from the beginning. The output also contains the images on which predictions were made.
Note: This model is built over a chunk of data. One can obtain the model for the entire data by running engine.py by taking the entire data to train the models.
4. Lib folder - This is a reference folder. It contains the original ipython notebook that we saw in the videos. The ppt used for explanation is also provided in this folder.

Project Takeaways

1. Understand the business context.
2. Understand Autoencoders.
3. Understand Transfer learning and backbone concept
4. Understand the VGG16 model architecture
5. Import the dataset and required libraries.
6. Use the VGG16 model
7. Rescale the images using ImageDataGenerator
8. Convert the RGB to LAB format
9. What is a sequential model?
10. Build a sequential model
11. Compile the model with suitable parameters
12. Fit the model and train on a certain number of epochs
13. Save the trained model that can be used later
14. Interpret the generated results
15. Making predictions on the test data using the trained model

